CS3990: Final Project – 50 points

Week 13-14

Develop the **Gamified Quiz** using Responsive Layout Principles to ensure your web- App adapts seamlessly to any screen size.

!!! The appearance and behavior of GUI elements should strictly comply the template (explained with screenshots and a video)

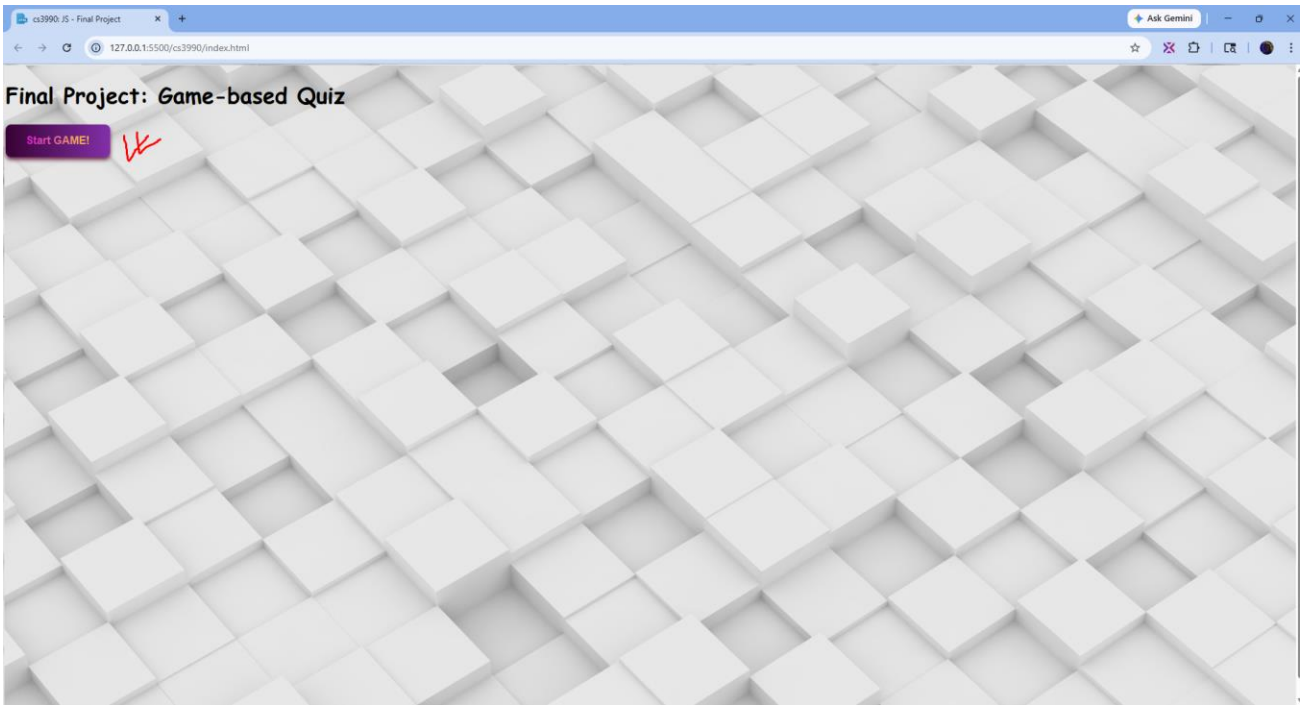
Technology Stack: *HTML, CSS, JS, jQuery*

Project Scope: a game-based quiz is an interactive browser application that dynamically displays questions, checks answers, updates results, and displays messages without reloading the page.

The Game uses 2 types of data:

- *quiz data* – used to display cards (with questions and answers,) and check the user choice;
- *assets* – set of individual elements (display as icons) that able to change the flow of game

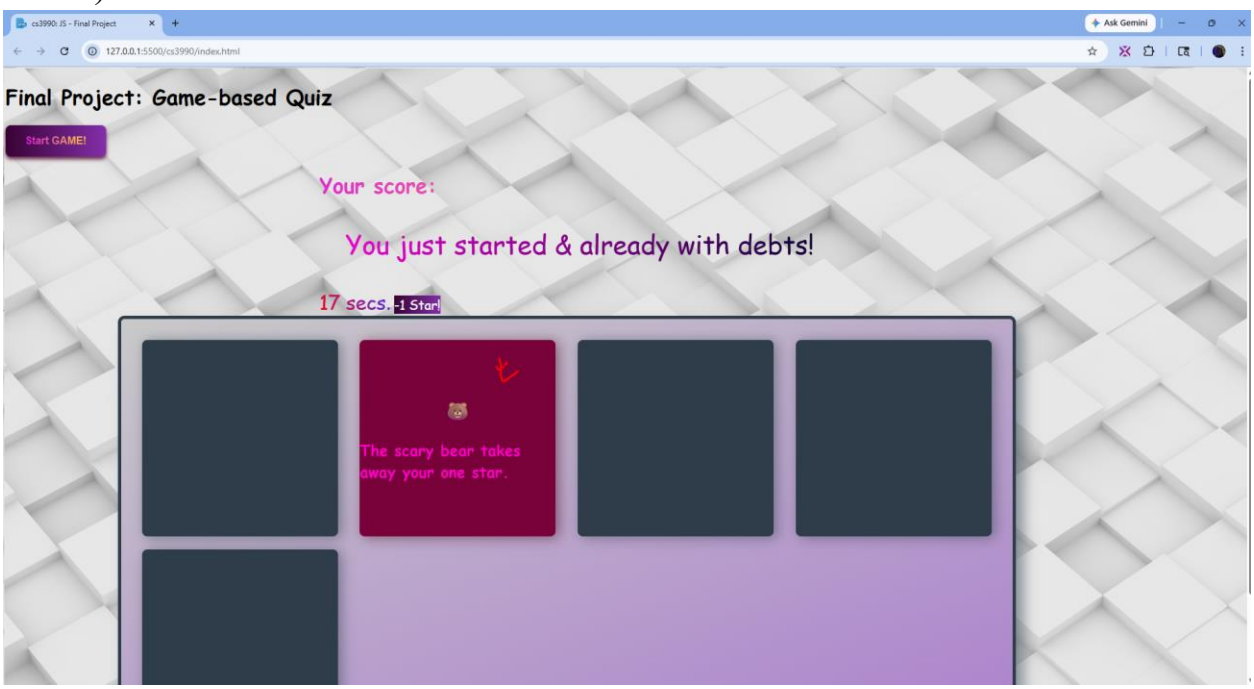
The playing field consists of a set of tiles, initially laid face down when you start the Game (by clicking on *Start Game* button)

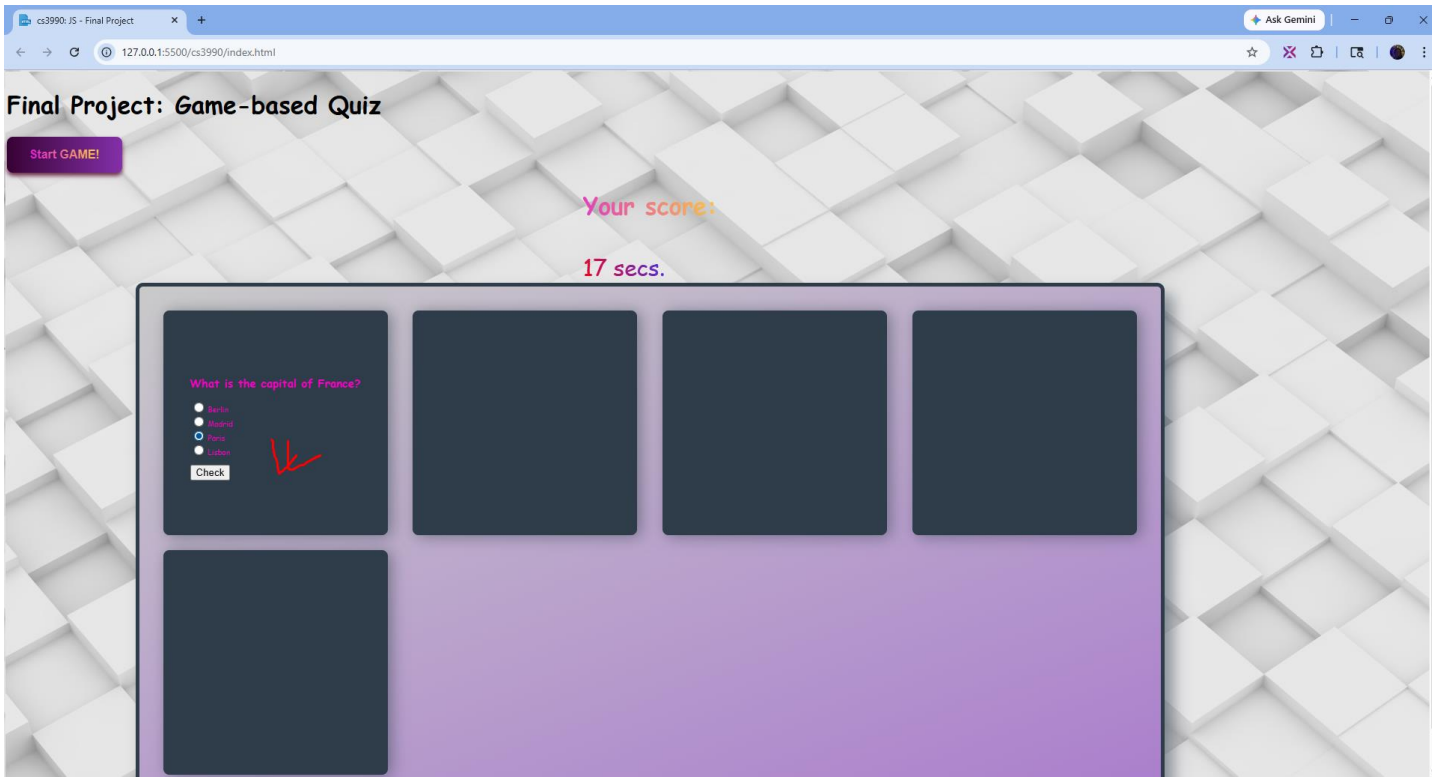


The number of tiles is equal to the total number of elements from both storages.

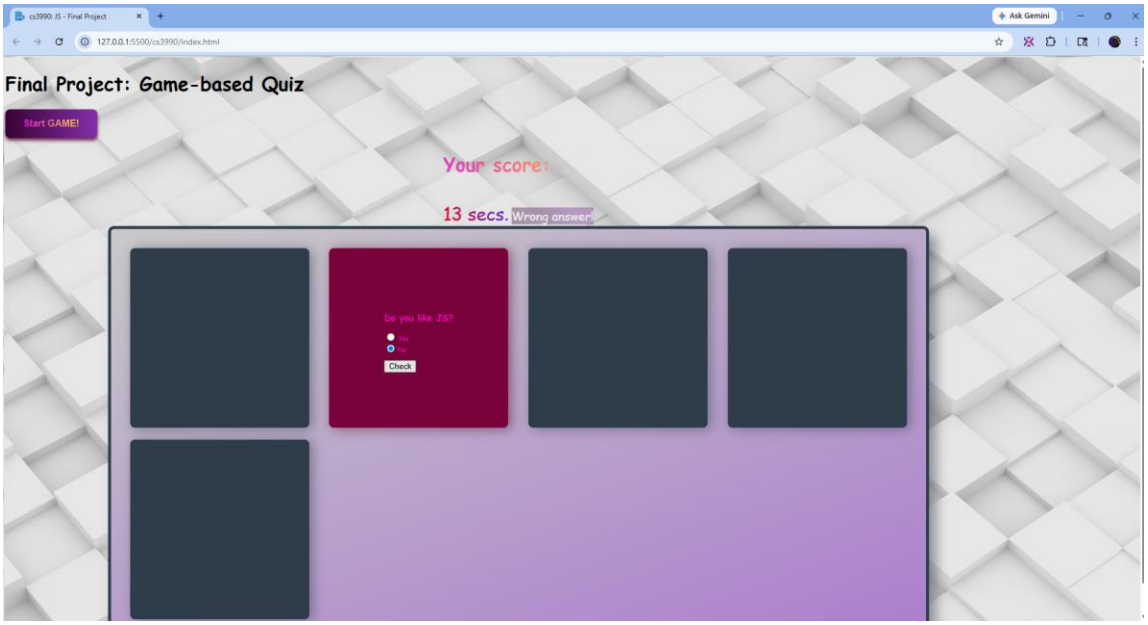
Details about App's logics and behavior

1. The Start button is the only graphical user interface element that is initially displayed on the screen (excluding the heading)
2. When you mouse over the Start Button its appearance is changed (explore the details in video)
3. When you click the "Start Game" button, the initial game Board appears, as well as the "Your Score" field and Timer Data.
4. The Timer starts immediately after the step above, and Timer Data is updated accordingly.
5. The User can click on any available tile (is not “DONE” - explained below)
6. The OnClick event rotates the tile and displays its contents (see screenshots below)





7. If the Tile is a Quiz Card, the user can select the answer (only one) and press *Check* button to check the correctness of the selected answer. If the answer is correct, a corresponding encouraging message appears, and it brings one point to the user (*Your score* field is updated immediately) If not -> the error.
8. For each point earned, a star is added (see screenshot below).
9. The appearance of the processed tile is changed (see screenshot below)



10. The message (encouraging or error) appears and disappears smoothly
11. If the user attempts to click the tile that has already been processed, a corresponding message will appear, and this will not affect the score in any way.

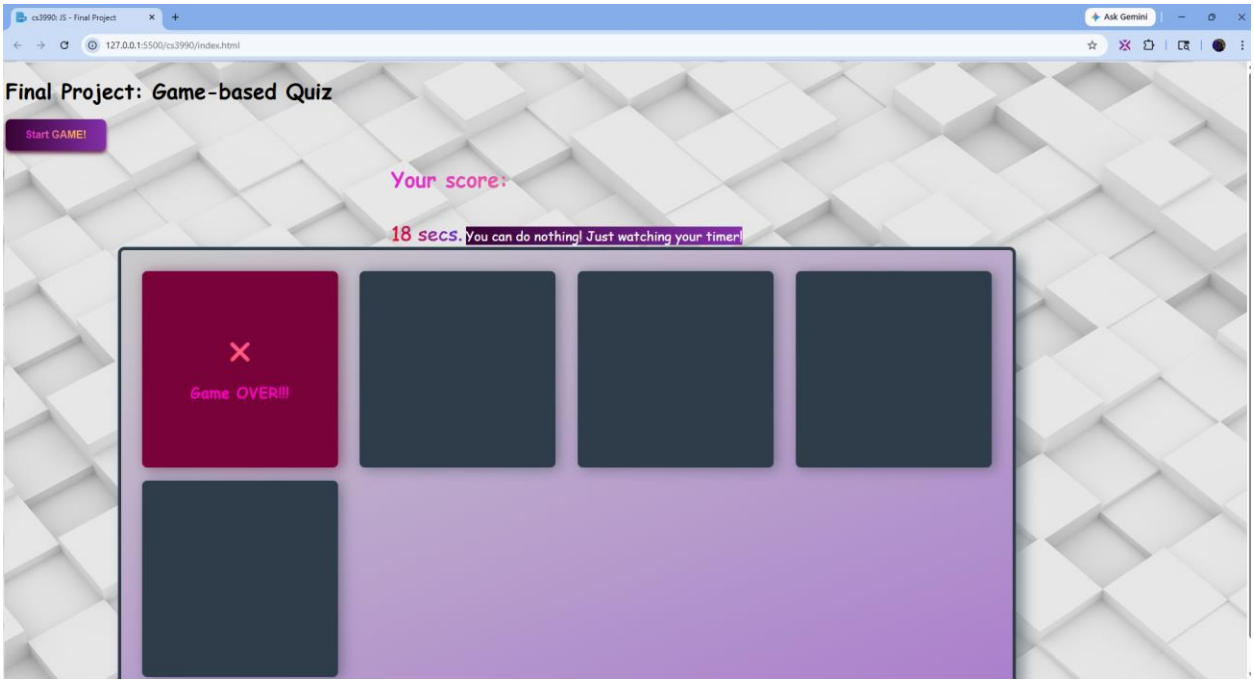
12. If the Tile is an Asset, it works as follows. And the score is updated accordingly

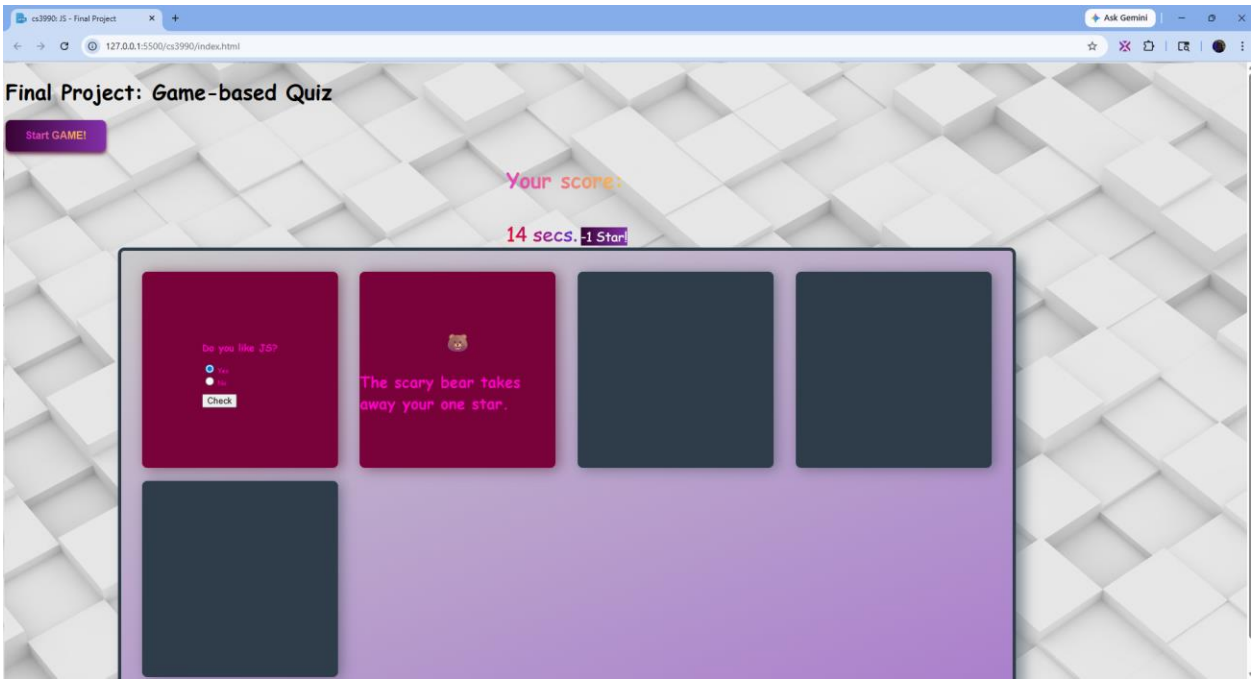
- The *Diamond* brings one star

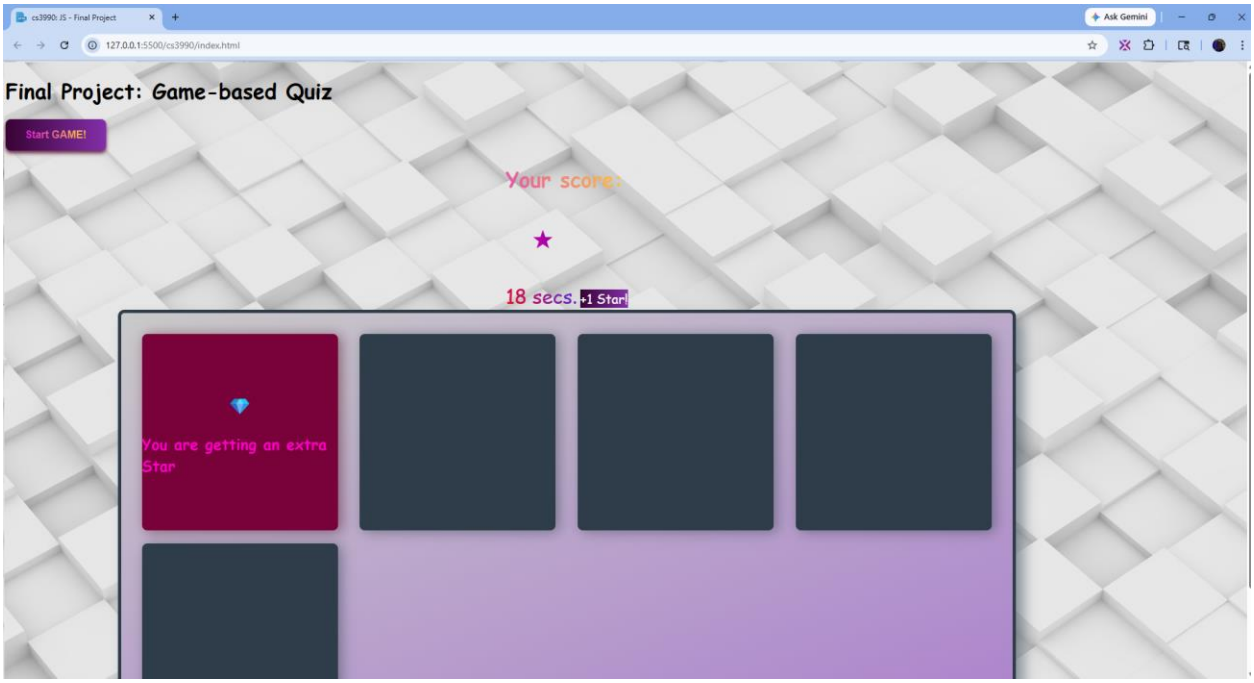
- The *Bear* takes away one star
- The C*ross* blocks the user's ability to continue working.

The messages (see screenshots below) appear and disappear smoothly

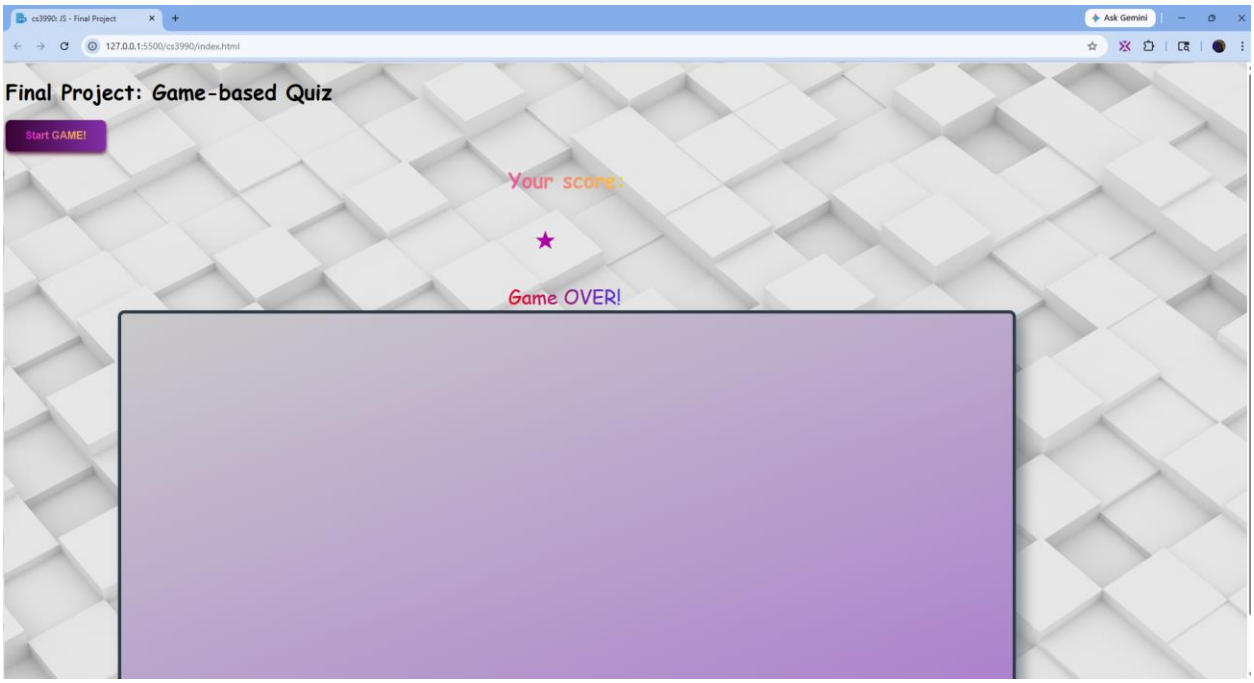
!!! Watch the video carefully to notice ALL the details and nuances regarding the appearance of the graphical interface and the game logic.







13.The game ends when the timer runs out, and the Board is cleared



!!! Project Tasks related to its implementation:

1. Implement data storage as 2 separate arrays: the 1st for quiz data and the 2ns for assets.

The minimum number of questions is 6, with half of them having at least 4 possible answers. Assets are already listed below:

```
const questions = [
        {
            question: "What is the capital of France?",
            options: ["Berlin", "Madrid", "Paris", "Lisbon"],
            correctAnswer: "Paris"
        },
        {question: "Do you like JS?",
            options: ["Yes","No"],
            correctAnswer: "Yes"        You, 2 days ago • final
        }

        // Add more questions here...
    ];

const assets=[
    {
        value:  "💎",
        descriptin: "You are getting an extra Star"
    },
    {
        value:  "🐻",
        descriptin: "The scary bear takes away your one star.
    },
        {
        value:  "❌",
        descriptin: "Game OVER!!!"
    }
    // Add more assets here...
];
```

Before each game, these 2 storages should be merged into one and shuffled.

2. Create and display the board – the number of tiles depends on the number of elements in the combined storage (*allCards*).

3. Create a component (as a class) *Card* to represent a Quiz Card with the following attributes and behavior:

Questions,

Options,

Correct answer;

render() method – to create appropriate elements that correspond required parts of a Quiz Card, wrap them with a common parent (a block)

show(container) method – to insert the block (created and filled with a render()) into DOM (the required place indicated via parameter - *container*)

The behavior of the Check Button should be described within a *Card* class since this button is a part of a card

```
this.checkBtn.onclick =(e)=>{ ···
}
```

!!! The card state is set to "Done" after the "Check" button is clicked, so this logic should be implemented in the `*this.checkbtn.onclick()*` method using the event object.

!!! <u>You can add additional attributes and/or methods into the Card class if necessary.</u>

4. Implement the countdown Timer by using a combination of *setInterval()* and *clearInterval()* methods. The duration of the Game is (*N_of_boardCards**2) seconds.

5. The Asset Card is created and displayed as a regular block during the *allCards* collection read cycle: however, if the current element is a Quiz Card, a new instance of the Card class is created and displayed.

6. Each card receives an event handler (onClick) after being added to the DOM. The OnClick event handler opens the front side of the card. However, if the card is an Asset, its state is set to "Done" after opening.

7. Create the following modules to implement the above tasks:

- Data.js (data storage with 2 arrays)
- Card.js (class Card)
- Utils.js (for additional and/or utility functions, like *timer(), arrShuffle()*, etc.)
- Main.js

8. To implement dynamic effects with smooth display, use the appropriate jQuery methods.

9. To create responsive layouts, use CSS Grid or CSS FlexBox. CSS media queries are allowed as well.

10. Use Font Awesome to get scalable vector icons for Assets, for ex.:

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-awesome.min.css">
```